

Secure Coding: The Dangers of Unhandled Errors

Lesson from Cloudflare Nov 18, 2025 Outage

Authored by: Douglas Mun, co-written using Google Gemini AI. **Dated:** Nov 19, 2025

Core Principle: Error Handling as a Security Boundary

In secure system design, **error handling goes beyond ensuring stability, it is a matter of security**. When a program encounters a condition it cannot manage, it must fail in a predictable and safe manner. Unpredictable failures, such as panicking or crashing, can create Denial-of-Service (DoS) vulnerabilities, allowing external input to disrupt the system.

Background: The Cloudflare Outage (Nov 18, 2025)

On November 18, 2025, at 11:20 UTC (all times in this blog are given in UTC), Cloudflare's network began experiencing significant failures in delivering core network traffic. Users attempting to access our customers' websites encountered an error page, indicating a failure within Cloudflare's network.

This issue was not a result of a cyberattack or any malicious activity. Instead, it was triggered by a change in the permissions of one of our database systems, which caused the database to generate multiple entries into a "feature file" used by our Bot Management system. As a result, this feature file doubled in size. The oversized feature file was then distributed to all the machines in our network.

The software running on these machines, responsible for routing traffic, reads this feature file to keep our Bot Management system updated with ever-changing threats. However, this software had a hard limit on the size of the feature file, which was lower than the doubled size. Consequently, this caused the software to fail.

The Cloudflare incident serves as a real-world example of failing to adhere to this principle in production.

- **Trigger:** A configuration change in a backend database led to the creation of an oversized "feature file" for the **Bot Management** system. This file contained **over 200 features**, exceeding an internal, hardcoded resource limit.
- **Vulnerable System:** The FL2 Rust proxy engine software was responsible for processing this file.
- **The Outcome:** When the oversized file was processed, the software did not gracefully reject the input. Instead, it crashed, resulting in widespread service disruption and demonstrating that a predictable, limited-resource condition could trigger a global outage.

The Code Flaw: Unsafe Error Handling in Rust

The vulnerability stemmed from using the `.unwrap()` method on the result of a fallible operation, essentially turning a recoverable error into a fatal system crash.

The Code Snippet

The core of the issue was in the `fetch_features` function:

Rust

```
pub fn fetch_features (
    // ... function signature ...
) -> Result<(), (ErrorFlags, 132)> {
    // ... setup code ...

    let (feature_values, _) = features
        .append_with_names(&self.config.feature_names)
        .unwrap (); // <--- CRITICAL FLAW: Panics on failure
}
```

The Coding Flaw Explained

1. **Fallible Operation:** The method `append_with_names()` is designed to check resource limits (such as the **200-feature limit**). If the limit is exceeded, it returns a **failure result** (`Err(E)`).
2. **The `.unwrap()` Method** on `Option` and `Result` types. It is used to extract the inner value of these types, but with a crucial caveat: if the `Option` is `None` or the `Result` is `Err`, calling `.unwrap()` will cause the program to **panic!** This method is a shortcut in Rust that says: “I guarantee this operation will succeed; if it fails, immediately **panic!** The current thread.”
3. **The Security Breach:** Because the oversized file caused `append_with_names()` to return `Err`, the subsequent `.unwrap()` triggered the panic.

The Security Impact

The use of `.unwrap()` on input that can be influenced by external systems (even indirectly via a configuration file) is a direct **violation of the security boundary** because it:

- **Converts a Validation Error into a Logic Error:** The system should have recognised the input as invalid and returned a controlled error. Instead, it treated the validation failure as an unrecoverable internal logic bug.
- **Enables Denial-of-Service (DoS):** Any user or internal process capable of generating a file larger than the limit can reliably crash the service, leading to service disruption.

Secure Coding Practise: Explicit Error Handling

The goal of secure error handling is to ensure that when a resource limit is exceeded, the program **returns a controlled error** to the caller rather than terminating the process. The recommended fix is to replace the fatal `.unwrap()` with the idiomatic Rust **? Operator** or a more explicit **match** statement.

The Secure Implementation (Using the ? Operator)

The `? try` or question mark operator, is a syntactic sugar used for error handling, ensures that if an operation returns an `Err`, the function immediately returns that error to its caller, preserving the thread's stability.

Rust

```
// Secure fix: Use '?' to gracefully propagate the error.
pub fn fetch_features (...) -> Result<(), (ErrorFlags, 132)> {
    // ...
    let (feature_values, _) = features
        .append_with_names(&self.config.feature_names)?; // <--- Fix:
        Propagates Err instead of panicking

    // This line is only reached if append_with_names was successful (Ok)
    Ok(())
}
```

Summary and Key Takeaways:

1. **Avoid Blind Unwrap:** Never use `.unwrap()` or `.expect()` on the result of any function that processes data crossing a trust boundary (network input, configuration files, database results).
2. **Explicit Handling is Security:** Use `match` or robust error-handling libraries to explicitly handle expected failure states such as resource limits, network failures, or invalid input.
3. **Fail Safe, Not Fatal:** A properly handled error returns control to the calling system, allowing the application to log the incident and continue serving other requests (e.g., returning a 400 Bad Request or 500 Internal Server Error) without crashing the entire worker thread.

Reference:

<https://blog.cloudflare.com/18-november-2025-outage/>

=== The End ===